

异常处理

北京理工大学计算机学院
金旭亮

概述

如果在处理入站事件的过程中有异常被抛出，那么它将从它在 `ChannelInboundHandler` 里被触发的那一点开始流经 `ChannelPipeline`。要想处理这种类型的入站异常，你需要在 `ChannelInboundHandler` 实现中重写下面的方法。

```
public void exceptionCaught(  
    ChannelHandlerContext ctx, Throwable cause) throws Exception
```

```
public class InboundExceptionHandler extends ChannelInboundHandlerAdapter {  
    @Override  
    public void exceptionCaught(ChannelHandlerContext ctx,  
        Throwable cause) {  
        cause.printStackTrace();  
        ctx.close();  
    }  
}
```

因为异常将会继续按照入站方向流动（就像所有的入站事件一样），所以实现了前面所示逻辑的 `ChannelInboundHandler` 通常位于 `ChannelPipeline` 的最后。这确保了所有的入站异常都总是会被处理，无论它们可能会发生在 `ChannelPipeline` 中的什么位置。

每个Channel都拥有一个与之相关联的ChannelPipeline，其持有一个ChannelHandler的实例链。在默认的情况下，ChannelHandler会把对它的方法的调用转发给链中的下一个ChannelHandler。因此，如果exceptionCaught()方法没有被该链中的某处实现，那么所接收的异常将会被传递到 ChannelPipeline 的尾端并被记录。为此，你的应用程序应该提供至少有一个实现了exceptionCaught()方法的ChannelHandler。

关于入站异常

`ChannelHandler.exceptionCaught()`的默认实现是简单地将当前异常转发给`ChannelPipeline`中的下一个`ChannelHandler`;

如果异常到达了`ChannelPipeline`的尾端,它将会被记录为未被处理;

要想定义自定义的处理逻辑,你需要重写`exceptionCaught()`方法。然后你需要决定是否需要将该异常传播出去。



处理出站异常

每个出站操作都将返回一个ChannelFuture。注册到ChannelFuture的 ChannelFutureListener 将在操作完成时被通知该操作是成功了还是出错了。

几乎所有的ChannelOutboundHandler上的方法都会传入一个ChannelPromise的实例。作为ChannelFuture的子类，ChannelPromise 也可以被分配用于异步通知的监听器。但是，ChannelPromise还具有提供立即通知的可写方法：

```
ChannelPromise setSuccess();  
ChannelPromise setFailure(Throwable cause);
```

添加ChannelFutureListener只需要调用ChannelFuture 实例上的 addListener (ChannelFutureListener)方法，并且有两种不同的方式可以做到这一点。其中最常用的方式是，调用出站操作（如 write()方法）所返回的ChannelFuture上的addListener()方法。

添加 ChannelFutureListener 到 ChannelFuture

```
ChannelFuture future = channel.write(someMessage);
future.addListener(new ChannelFutureListener() {
    @Override
    public void operationComplete(ChannelFuture f) {
        if (!f.isSuccess()) {
            f.cause().printStackTrace();
            f.channel().close();
        }
    }
});
```

第二种方式是将 `ChannelFutureListener` 添加到即将作为参数传递给 `ChannelOutboundHandler` 的方法的 `ChannelPromise`

```
public class OutboundExceptionHandler extends ChannelOutboundHandlerAdapter {
    @Override
    public void write(ChannelHandlerContext ctx, Object msg,
        ChannelPromise promise) {
        promise.addListener(new ChannelFutureListener() {
            @Override
            public void operationComplete(ChannelFuture f) {
                if (!f.isSuccess()) {
                    f.cause().printStackTrace();
                    f.channel().close();
                }
            }
        });
    }
}
```